

Google Pay API 網頁版 101：基本概念

來源：<https://codelabs.developers.google.com/codelabs/gpay-web-101?hl=zh-tw>

步驟數：7

目錄

1. [1. 簡介](#)
2. [2. 建立 HTML 網頁](#)
3. [3. 設定 Google Pay](#)
4. [4. 新增付款用戶端](#)
5. [5. 新增幫手](#)
6. [6. 新增事件處理常式](#)
7. [7. 結論](#)

步驟 1 · 約 3 分鐘

1. 簡介

建構項目

完成本程式碼研究室後，您將擁有可正常運作的網站，並整合 Google Pay。這個專案會擷取付款權杖，並傳送給付款服務供應商進行處理。

課程內容

- 如何載入及設定 Google Pay API
- 如何顯示 Google Pay 按鈕及處理點擊事件
- 如何向 Google Pay 申請付款權杖

軟硬體需求

- 您選擇的文字編輯器，用於編輯 HTML 和 JavaScript 檔案。
 - Google Chrome 網路瀏覽器，或測試本機網站的其他方式。
 - 如要使用正式版，您需要 Google Pay `merchantId`。在 [Google Pay](#) 和 [錢包主控台](#) 註冊只需一分鐘，建議您現在就完成註冊。
-

2. 建立 HTML 網頁

建立專案檔案

1. 在電腦上建立名為 `gpay-web-101` 的資料夾，並在該資料夾中建立兩個空白文字檔，分別命名為 `index.html` 和 `main.js`。目錄結構應如下所示：

```
gpay-web-101/  
  index.html  
  main.js
```

2. 在慣用的 IDE 中開啟 `index.html`，然後新增下列程式碼：

```
<!doctype html>  
<html lang="en">  
  
<head>  
  <meta charset="UTF-8" />  
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />  
  <title>Google Pay API for Web 101</title>  
</head>  
  
<body>  
  <div id="gpay-container"></div>  
  <p>Transaction info and errors will be logged to the console.</p>  
  <script type="text/javascript" src="main.js"></script>  
  <script  
    async src="https://pay.google.com/gp/p/js/pay.js"  
    onload="onGooglePayLoaded()">  
  </script>  
</body>  
  
</html>
```

解釋程式碼

1. 系統會在頁面中新增 ID 為 `gpay-container` 的空白 DIV。這個 DOM 元素會是新增 Google Pay 按鈕的父項元素。您可以視需要將這個元素放在網站版面配置中。
 2. `main.js` 指令碼標記會放在 DOM 中的 `gpay-container` 元素之後。這是必要步驟，可確保容器 DIV 存在於 DOM 中，`main.js` 才會查詢該容器。此外，指令碼是同步的，可確保在 `pay.js` 載入前載入，因為 `onGooglePayLoaded()` 方法必須在載入完成前存在。還有其他方法可以達到相同效果，但這裡不會討論。
 3. 最後，系統會以非同步方式載入 `pay.js`，並將 `onGooglePayLoaded()` 設定為 `onload` 處理常式。這個方法將在 `main.js` 中定義。
-

步驟 3 · 約 5 分鐘

3. 設定 Google Pay

Google Pay 付款要求需要要求物件。這裡定義的物件 `baseGooglePayRequest` 包含所有要求的最低通用設定。我們會根據提出的要求新增其他設定，並在本程式碼研究室中進行審查。

在空白的 `main.js` 檔案中新增 Google Pay 設定常數：

```
//=====
// Configuration
//=====

// The DOM element that the Google Pay button will be rendered into
const GPAY_BUTTON_CONTAINER_ID = 'gpay-container';

// Update the `merchantId` and `merchantName` properties with your own values.
// Your real info is required when the environment is `PRODUCTION`.
const merchantInfo = {
  merchantId: '12345678901234567890',
  merchantName: 'Example Merchant'
};

// This is the base configuration for all Google Pay payment data requests.
const baseGooglePayRequest = {
  apiVersion: 2,
  apiVersionMinor: 0,
  allowedPaymentMethods: [
    {
      type: 'CARD',
      parameters: {
        allowedAuthMethods: [
          "PAN_ONLY", "CRYPTOGRAM_3DS"
        ],
        allowedCardNetworks: [
          "AMEX", "DISCOVER", "INTERAC", "JCB", "MASTERCARD", "VISA"
        ]
      },
      tokenizationSpecification: {
        type: 'PAYMENT_GATEWAY',
        parameters: {
          gateway: 'example',
          gatewayMerchantId: 'exampleGatewayMerchantId'
        }
      }
    }
  ],
  merchantInfo
};

// Prevent accidental edits to the base configuration. Mutations will be
// handled by cloning the config using deepCopy() and modifying the copy.
Object.freeze(baseGooglePayRequest);
```

解釋程式碼

1. 將常數變數 `GPAY_BUTTON_CONTAINER_ID` 設為 HTML 網頁上使用的 DOM 元素 ID，做為 Google Pay 按鈕的上層容器。

2. 為應用程式建立包含相關設定的設定物件 `baseGooglePayRequest`。您可以在[參考說明文件](#)中找到各項屬性和值。這個範例中顯示的值可能完全符合您的需求，也可能不符，因此請仔細檢查。
3. 使用您自己的值更新 `merchantId` 和 `merchantName` 屬性。如果環境為 `TEST`，這些欄位為選填。

資源

- 網誌文章：[想透過 Google Pay 更順暢地結帳嗎？設定付款選項！](#)
- **API 參考資料：**Google Pay API [要求物件](#)說明文件
- **API 參考資料：**如要進一步瞭解允許的授權方法、允許的卡片網路和代碼化規格 (包括適當的閘道值)，請參閱 [PaymentMethod](#)。

在 `TEST` 環境中，`merchantId` 和 `merchantName` 為選填欄位。不過，`PRODUCTION` 環境會要求提供這項資訊。請前往 [Google Pay 和錢包主控台](#)，立即取得。

4. 新增付款用戶端

付款用戶端可用於提出付款要求及註冊回呼。在本程式碼研究室中，我們只會提出付款要求。此外，您也可以設定 `PaymentDataCallbacks`，以便在付款資料或授權變更時進行處理。不過，本程式碼研究室不會涵蓋這些進階主題。

將這段用戶端程式碼附加至 `main.js` 底部：

```
//=====
// Google Payments client singleton
//=====

let paymentsClient = null;

function getGooglePaymentsClient() {
  if (paymentsClient === null) {
    paymentsClient = new google.payments.api.PaymentsClient({
      environment: 'TEST',
      merchantInfo,
      // todo: paymentDataCallbacks (codelab pay-web-201)
    });
  }

  return paymentsClient;
}
```

解釋程式碼

1. 建立執行個體後，`paymentsClient` 變數會將執行個體保留在用戶端。我們的程式碼不會直接存取變數，而是會一律透過 `getGooglePaymentsClient()` 方法存取。
 2. `getGooglePaymentsClient()` 方法會檢查是否已例項化用戶端，並傳回該例項。如果先前未例項化，系統會建立、儲存並傳回例項。這種方法可確保在整個指令碼生命週期內，只會建立及使用一個執行個體。
 3. 如要例項化用戶端，請叫用 `PaymentsClient()` 方法。在本例中，我們告知用戶端目前處於 `TEST` 環境。替代方案是 `PRODUCTION`。不過，`TEST` 是預設值，可以省略。
-

5. 新增幫手

指令碼稍後會用到下列輔助函式，加入這些函式的唯一目的是提升程式碼的可讀性和可維護性。

將輔助函式附加至 `main.js` 的底部：

```
//=====
// Helpers
//=====

const deepCopy = (obj) => JSON.parse(JSON.stringify(obj));

function renderGooglePayButton() {
  const button = getGooglePaymentsClient().createButton({
    onClick: onGooglePaymentButtonClicked
  });

  document.getElementById(GPAY_BUTTON_CONTAINER_ID).appendChild(button);
}
```

解釋程式碼

1. `deepCopy()` 是一種公用程式，可使用 JSON 序列化和還原序列化，建立所提供物件的深層副本。這是複製物件的便利方式，不必擔心共用參照。
2. `renderGooglePayButton()` 是一種公用程式，可叫用 `createButton()` 程式庫方法來顯示 Google Pay 按鈕。傳遞的引數是一組選項，用於定義按鈕的行為，例如註冊 `onGooglePaymentButtonClicked()` 函式來處理按鈕點擊事件。

如要進一步瞭解按鈕樣式選項，請參閱「[自訂按鈕](#)」指南。

步驟 6 · 約 3 分鐘

6. 新增事件處理常式

在這個指令碼中，我們設定了兩個事件處理常式。第一個會在 `pay.js` 程式庫載入完成時呼叫，另一個則會在點選 Google Pay 按鈕時呼叫。

將事件處理常式附加至 `main.js` 底部：

```
//=====
// Event Handlers
//=====

function onGooglePayLoaded() {
  const req = deepCopy(baseGooglePayRequest);

  getGooglePaymentsClient()
    .isReadyToPay(req)
    .then(function(res) {
      if (res.result) {
        renderGooglePayButton();
      } else {
        console.log("Google Pay is not ready for this user.");
      }
    })
    .catch(console.error);
}

function onGooglePaymentButtonClicked() {
  // Create a new request data object for this request
  const req = {
    ...deepCopy(baseGooglePayRequest),
    transactionInfo: {
      countryCode: 'US',
      currencyCode: 'USD',
      totalPriceStatus: 'FINAL',
      totalPrice: (Math.random() * 999 + 1).toFixed(2),
    },
  },
  // todo: callbackIntents (codelab gpay-web-201)
};

// Write request object to console for debugging
console.log(req);

getGooglePaymentsClient()
  .loadPaymentData(req)
  .then(function (res) {
    // Write response object to console for debugging
    console.log(res);
    // @todo pass payment token to your gateway to process payment
    // @note DO NOT save the payment credentials for future transactions
    paymentToken = res.paymentMethodData.tokenizationData.token;
  })
  .catch(console.error);
}
```

解釋程式碼

1. 當 `pay.js` 指令碼完成載入 (如 HTML 檔案中所定義) 時，系統會叫用 `onGooglePayLoaded()`。系統會叫用 `isReadyToPay()` 方法，判斷是否顯示 Google Pay 按鈕。如果消費者已準備好付款 (也就是已在 Google 錢包中新增付款方式)，系統就會顯示 Google Pay 按鈕。
2. 點選 Google Pay 按鈕時，系統會叫用 `onGooglePaymentButtonClicked()`。這個方法會叫用 `loadPaymentData()` 程式庫方法，用於擷取付款權杖。取得付款權杖後，請將權杖傳送至支付閘道，以處理交易。`transactionInfo` 說明應透過點選這個按鈕處理的交易。

如要進一步瞭解如何為交易設定要使用的權杖化類型，請參閱 `tokenizationData`。

7. 結論

恭喜您完成本程式碼研究室！您已學會如何將 Google Pay API 整合至網站。

執行專案

使用 Google Chrome 測試

使用 **Google Chrome** 網路瀏覽器開啟 `index.html`，方法是依序點選 Chrome 主選單中的「檔案」>「開啟檔案...」。以這種方式開啟專案時，Chrome 會執行 `main.js`。其他網路瀏覽器可能不允許執行 JavaScript。

– 或 –

使用本機網路伺服器進行測試

如果已安裝 Python，請在根 `pay-web-101` 資料夾的終端機提示中執行 `python3 -m http.server`。

```
$ cd /your/path/to/pay-web-101
$ python3 -m http.server
Serving HTTP on :: port 8000 (http://[::]:8000/) ...
```

然後前往 `http://localhost:8000` 存取網站。

下一步該做什麼？

- [Google for Web 201：進階](#)
- [自訂 Google Pay 按鈕](#)
- [查看整合檢查清單](#)

其他資源

- 在 Discord 的 [#payments](#) 頻道中加入對話
 - 在 X 上追蹤 [@GooglePayDevs](#)
 - 在 YouTube 觀看 [Google Pay](#) 相關影片
-